PYTHON PROGRAMMING | COURSE PROJECT

Number Utility Toolkit

Project report for Group 8

> Objective
> Build a menu-driven program that checks numbers and performs common number calculations.

Group members

Rajesh Chaurasiya

Manas Dubey

Sweta

Pavan Kumar

Prepared for the Python Programming course

The contribution table on the final page should be checked by the group and updated to match the work each person actually completed.

1. Introduction and objectives

The Number Utility Toolkit brings several small number operations into one menu. A user selects a tool, enters whole-number values, and reads the result on the screen. The program stores completed results in a text file so they can be viewed again in a later run.

The project gives practice with functions, conditions, loops, collections, recursion, file handling, exceptions and doctests. It is written as one Python file and uses only the Python standard library.

Objectives

• Check prime, palindrome, Armstrong and perfect numbers.

• Calculate digit details, factorial, Fibonacci values, GCD and LCM.

• List primes and multiplication tables up to a user-selected limit.

• Handle wrong input without stopping the menu.

• Save completed results and load the history next time.

Problem statement

A learner may need to switch between different small programs to check properties of numbers or calculate results. This project combines those operations in one simple menu. It validates user input, limits large recursive calculations, and keeps a text record of results.

Scope

The program accepts whole numbers. Factorial and Fibonacci inputs are limited to 0-20, prime listing is limited to 2-500, and multiplication tables are limited to 1-12. It is a learning project; it does not use a database or graphical interface.

2. Program design and algorithm

The main function loads the result history and then repeats the menu inside a while loop. Each menu option collects the values it needs and calls a separate function. After a calculation, the result is displayed and appended to the text file.

Main flow

Step	Action
1	Start the program and try to load number_utility_results.txt. If it is missing, begin with an empty history.
2	Display the menu and ask for a whole-number choice. Re-prompt if the input is blank or contains letters.
3	Check that the choice is from 1 to 12. For calculation choices, ask for the required number or pair of numbers.
4	Check input limits where needed, then call the function for that calculation.
5	Display the result, store it in the results dictionary and append a readable line to the text file.
6	Repeat the menu. Choice 11 shows recent saved results; choice 12 exits.

Simplified flow

Start	Load history	Show menu	Validate input	Calculate + save	Repeat / exit

The menu returns to the same point after each calculation. Invalid text input is handled in read_number(), while out-of-range values return to the menu with an explanation.

3. Features and how to use the menu

Run the program from the project folder with python main.py or, on some Windows computers, py main.py. Enter the number beside a menu option and then follow its prompts.

Menu options

Choice	Tool	What it does / input range
1	Prime check	Checks whether one integer is prime. Values below 2 are not prime.
2	Palindrome check	Checks whether the digits read the same in reverse.
3	Armstrong check	Checks whether the sum of each digit raised to the digit count equals the input.
4	Perfect number check	Checks whether proper positive divisors add up to the input.
5	Digit details	Returns digit sum, digit count and reversed value.
6	Fibonacci series	Asks for 0-20 values and returns the series.
7	Factorial	Calculates factorial for an input from 0 to 20.
8	GCD and LCM	Asks for two integers and returns both results.
9	List primes	Lists prime numbers from 2 up to a limit of 2-500.
10	Multiplication tables	Prints tables from 1 up to a number from 1-12.
11	Recent results	Shows up to five recent saved results.
12	Exit	Ends the menu loop.

Example: choose 3, enter 153, and the program displays Armstrong Check: True. The result is also written to number_utility_results.txt.

4. Function descriptions

The program is organised as small functions so each calculation can be read and doctested separately. Every function has a short comment and a docstring with examples or a basic doctest check.

Number checks and calculations

Function	Purpose
is_prime(number)	Tests divisors up to the square root of the number.
is_palindrome(number)	Compares the number string with its reversed slice.
is_armstrong(number)	Raises each digit to the number of digits and adds the values.
is_perfect(number)	Adds proper divisors and compares the sum with the number.
digit_details(number)	Returns digit sum, digit count and reverse as a tuple.
factorial(number)	Uses recursion and stops at 0 or 1.
fibonacci_value(index)	Returns one Fibonacci value through recursion.
fibonacci_series(count)	Returns a tuple of Fibonacci values.
gcd(first, second)	Uses the recursive Euclidean algorithm.
gcd_lcm(first, second)	Returns GCD and LCM as a tuple.
primes_up_to(limit)	Uses nested loops and a set to find unique primes.
multiplication_tables(limit)	Uses nested loops to build table lines.

Menu and file functions

Function	Purpose
format_result(operation, value)	Cleans the operation label and formats a result line.
load_history()	Reads earlier result lines and handles a missing file.
save_result(line, history)	Adds a result to the list and appends it to the file.
show_recent(history)	Slices the most recent five entries, sorts them and returns a tuple.
read_number(prompt)	Repeats a prompt until a whole number is entered.
main()	Runs the menu and connects the tools.

5. Python course topics used

The code uses the required course topics in ways that support the number tools and the menu.

Topic map

Topic	Where it appears
Variables and data types	Integers hold input values and calculation results. Strings hold prompts, labels and saved lines. This project uses integer arithmetic because its operations are number checks and whole-number calculations.
Strings	Input uses strip(); result labels use lower(), split(), join() and title(); read_number() checks isdigit().
Operators	Arithmetic operators calculate digit powers, factorial, remainders and LCM. Comparisons and logical and/or/not checks support validation. The in/not in membership check validates menu choices.
Lists	The history list uses append(), remove(), slicing and sort(). A list also stores multiplication-table lines.
Dictionary	The results dictionary stores calculation names and values during the current run.
Set	A set stores primes found up to N, preventing duplicate values.
Tuple	Several functions return fixed groups of values, such as digit_details() and gcd_lcm().
Conditions	if / elif / else selects a tool, applies number rules and handles range limits.
Loops	The menu uses while; for loops inspect digits and values. Nested loops search primes and build multiplication tables.
Functions and recursion	Calculation work is divided into functions. Factorial, Fibonacci values and GCD call themselves with base cases.
Files, exceptions and doctests	A .txt file stores results. try/except handles missing files and calculation errors. Doctest examples are checked with python -m doctest -v main.py.

The program uses only built-in Python features and the standard doctest module. No external packages are needed to run it.

6. Recursion, nested loops and collections

Recursion

The recursive functions reduce the problem each time and stop at a base case. Without the base case, a function would keep calling itself until Python reports an error.

```
factorial(n):
    if n <= 1: return 1
    return n * factorial(n - 1)

fibonacci(n):
    if n <= 1: return n
    return fibonacci(n - 1) + fibonacci(n - 2)

gcd(a, b):
    if b == 0: return a
    return gcd(b, a % b)
```

For factorial(4), the calls reduce from 4 to 3 to 2 to 1; the base case returns 1, then the results multiply back to 24. Fibonacci inputs are capped at 20 because this simple recursive version repeats work.

Nested loops

In primes_up_to(), the outer loop visits candidates and the inner loop checks possible divisors. In multiplication_tables(), the outer loop chooses the table number and the inner loop creates its rows.

Collections

The results dictionary links operation names to values. The set makes prime results unique. Tuples keep fixed output groups together, such as (digit sum, digit count, reverse) and (GCD, LCM). The history list can change as new lines are added.

7. Testing and results

The function examples were run with Python doctest. The interactive program was also run with normal input, invalid text, calculations and a restart to check that results could be loaded again.

Test cases

No.	Input / action	Expected result	Observed result
1	Check Armstrong number with 153	True	True
2	Check perfect number with 28	True	True
3	GCD and LCM for 24 and 18	(6, 72)	(6, 72)
4	List primes up to 10	(2, 3, 5, 7)	(2, 3, 5, 7)
5	Digit details for 408	(12, 3, 804)	(12, 3, 804)
6	Type letters at a number prompt, then enter 23	Show a clear message, accept 23, then show Prime Check: True	Message shown; 23 accepted; Prime Check: True
7	Run once, save a result, restart and choose 11	Earlier result appears in recent history	Saved result loaded and displayed

Doctest command

```
python -m doctest -v main.py
```

The doctest command imports the module and checks the examples in the function docstrings without starting the menu. The saved results file may be read while checking load_history().

Testing note: these checks cover the listed examples and menu paths. They do not prove that every possible input is error-free.

8. Sample program runs

These terminal-style captures show outputs from the program runs. The first shows a successful number check; the second shows that invalid text is rejected and the program asks again.

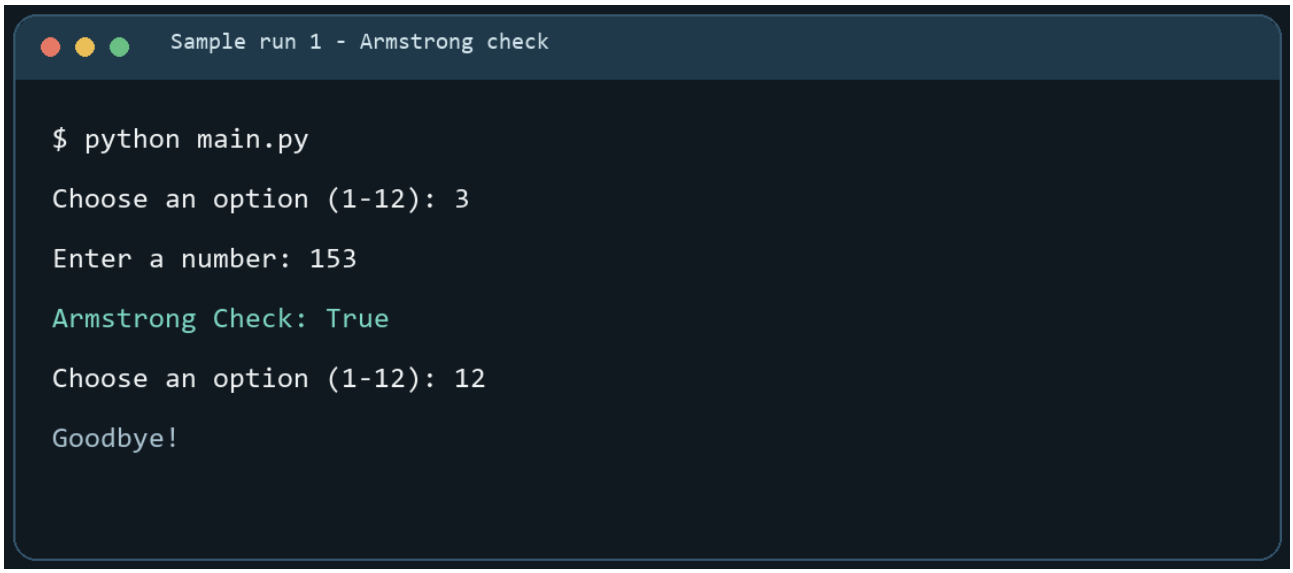


Figure 1. Example run: 153 is reported as an Armstrong number.

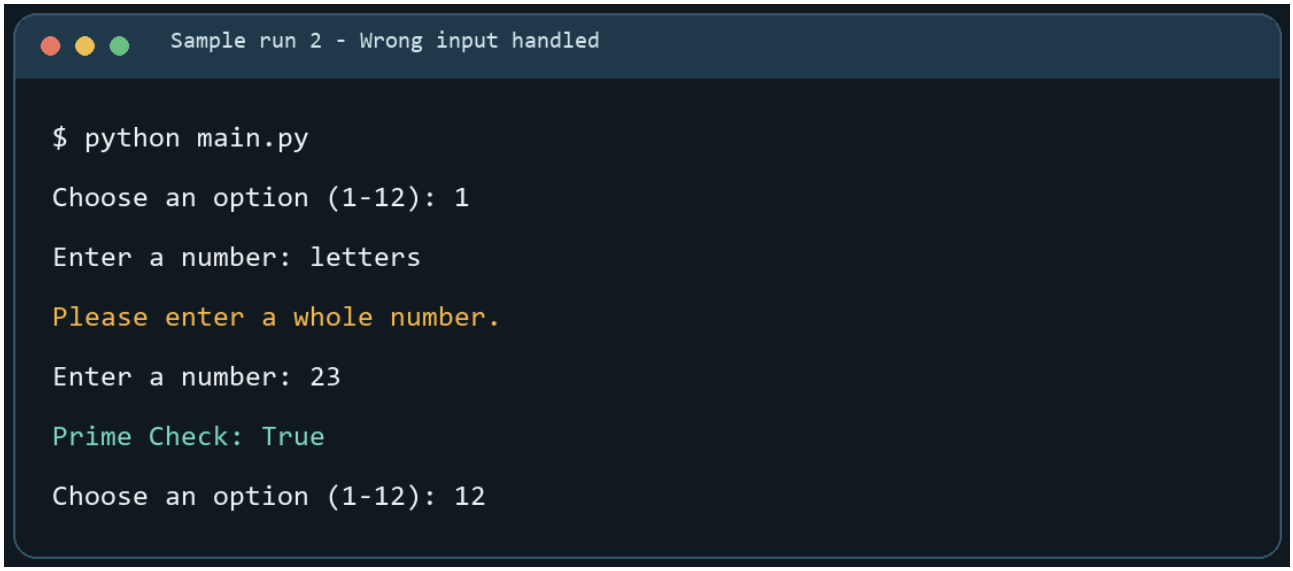


Figure 2. Example run: letters are rejected and the program accepts the next whole number.

9. Saving and loading results

After each calculation, save_result() appends one formatted line to number_utility_results.txt. At startup, load_history() reads existing non-empty lines. If the file is missing on a fresh computer, the program starts with an empty history; the first saved calculation creates the file.

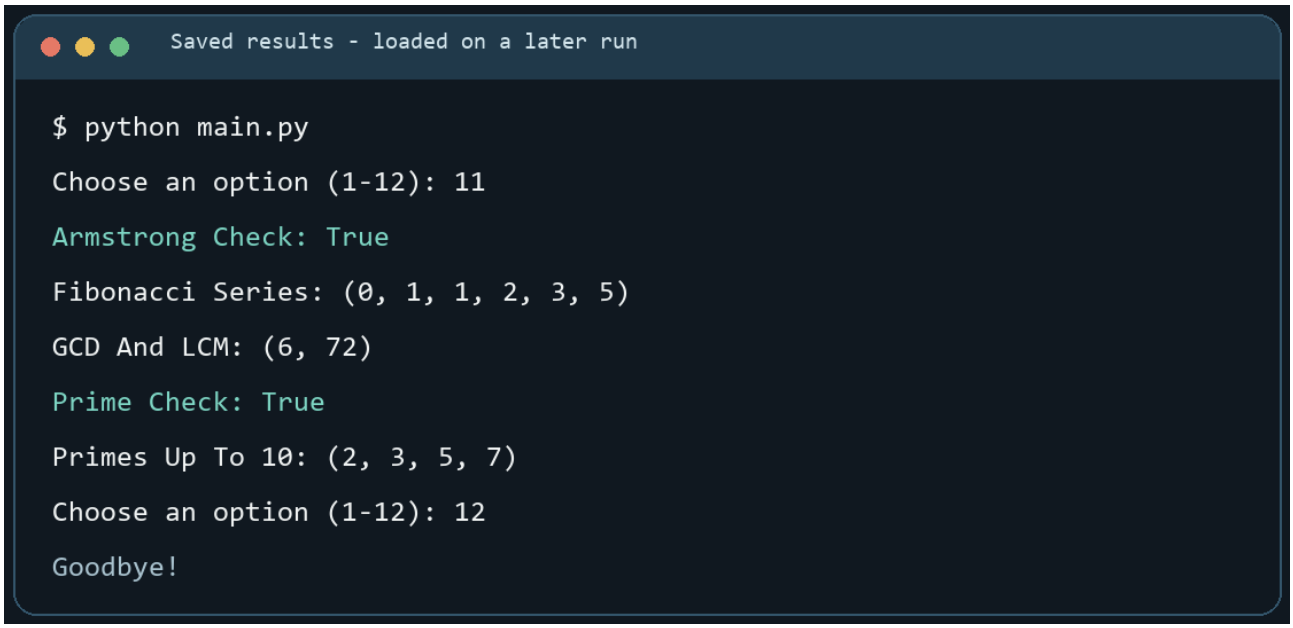


Figure 3. Example run: option 11 displays up to five recent saved results.

The results file is plain text and can be opened in any text editor. A copy of it is included with the source code so the project folder can be run and reviewed together.

Run on a fresh computer

Keep main.py and number_utility_results.txt in the same folder. Open a terminal in that folder and run python main.py. The program does not require any packages to be installed.

10. Conclusion and contribution record

Conclusion

The project combines number checks and calculations in one menu. It demonstrates separate functions, loops, nested loops, recursion, collections, input validation, doctests and text-file handling. The program can be extended with more tools while keeping the same menu structure.

Limitations and possible improvements

• Only whole-number input is supported.

• Factorial and Fibonacci are limited to 20, prime listing to 500, and tables to 12.

• The program does not provide a clear-history or export option.

• Future versions could add a clear-history menu item, decimal tools where appropriate, and more detailed input help.

Group contribution table

The brief asks each group to record who did what. Fill in the last column with the actual contribution before submission; do not leave these prompts as-is.

Member	Contribution
Rajesh Chaurasiya	Confirm and enter actual work completed
Manas Dubey	Confirm and enter actual work completed
Sweta	Confirm and enter actual work completed
Pavan Kumar	Confirm and enter actual work completed

Source basis

Python Project Briefs for Students, Project 8: Number Utility Toolkit; Python Project Evaluation Guidelines. The project name, topics and required tools in this report follow the Group 8 brief.